CSC 665 Assignment 2 - Written Answers

Nathaniel Moreno
Enjun Li

Problem I:

Part 1b – Implementation (MCTS)

- Each node in the MCTS tree represents a game state in the Coins in a Line game. A node stores: the current state, the parent node, the action that led to this node, its children, a list of untried actions, the number of visits (N), the total reward/value (W).
  Visits and values are updated during the backpropagation phase of MCTS.
- When expanding a node, we choose one of the actions in untried_actions. The implementation randomly selects one action and removes it from the list, then creates a child node for that action.
- The rollout policy used is a random policy. During rollout, the algorithm repeatedly selects a random legal action until the game reaches a terminal state. (random.choice(actions(s))

Part 2 – Experiments and Comparisons

=== Reward mode: winloss ===

Budget	Games	Win%	Loss%	Tie%	AvgDiff	MCTS ms/mv	MM ms/mv
20	60	93.3%	6.7%	0.0%	13.27	0.20	488.24
200	60	98.3%	1.7%	0.0%	16.97	1.21	298.06
2000	60	96.7%	3.3%	0.0%	17.47	11.88	298.48
20000	60	90.0%	6.7%	3.3%	16.43	116.74	306.74

Part 3 – Result Interpretation and Analysis

(1a)
As the MCTS budget increases, performance generally improves at first. The win rate increases from 93.3% at budget 20 to 98.3% at budget 200, and the average score difference also increases. However, at larger budgets (2000 and 20000), the win rate decreases slightly. This suggests that while additional simulations help MCTS make better decisions, the improvement eventually shows diminishing returns and some variability due to the randomness of rollouts.

(1b)
The move computation time for MCTS increases roughly linearly with the budget. For example, the average time per move increases from 0.20 ms at budget 20 to 116.74 ms

at budget 20000. This is expected because MCTS performs one simulation per iteration, so more iterations directly increase computation time. In contrast, minimax time remains relatively constant across experiments.

(2a)
MCTS beating minmax is a logical outcome because in minmax's implementation:

```
if terminal(state):
    p1, p2 = utility(state)
    return (p2 - p1 if is_maximizing else p1 - p2), None
```

This line actually shows that the score switches between the scores of AI - Player and Player - AI based on is_maximizing.

(2b)
However, in MCTS, it always focuses on the root player's perspective. This means that it does not have the problem of switching the perspectives, and by having a single consistent perspective, MCTS can accurately decide which moves are actually "good".

(3)
One reason MCTS may still struggle is because it uses a random rollout policy during the simulation phase. Since the rollouts choose moves randomly, the simulation results can contain a lot of noise. Even with a large number of simulations, this randomness can lead to inaccurate estimates of move quality.

This can be seen in the results. If larger budgets always improved performance, we would expect the win rate to increase steadily. However, the win rate increases from 93.3% at budget 20 to 98.3% at budget 200, but then decreases to 96.7% at budget 2000 and 90.0% at budget 20000. This variation suggests that the random rollout policy can still introduce instability in the results.

Problem II: KB definition (Knights and knaves)
For each puzzle we coded: (1) puzzle structure- each character is exactly one of the knight or knave, (2) What was said- if x says s, then (XKnight -> S) Λ (Xknave -> ¬S). The model checker then finds which symbols are entailed.

Puzzle 1:
Statement: A says, "I am both a knight and a Knave."
Structure: A is exactly one of knight or knave: Aknight ⊕ AKnave.

What A said: S = “I am both a knight and a knave” = Aknight ∧ AKnave.
Encoding “A says S”: If A is a knight then S is true: AKnight ⇒ (AKnight AKnave). If A is a Knave then S is false: AKnave ⇒ ¬(AKnight C AKnave).

KB: KB_1 = (AKnight ⊕ AKnave) ∧ (AKnight ⇒ (AKnight ∧ AKnave)) ∧ (AKnave ⇒ ¬(AKnight ∧ AKnave)).

Result: The only model satisfying KB_1 has AKnave true (and AKnight false). So **A is a knave** — the statement is impossible for a knight and is a lie for a knave.

Puzzle 2:
(characters A and B)

Statements: A says “We are the same kind.” B says “We are different kinds.”

-Structure: A and B each exactly one of knight/knave: (AKnight ⊕ AKnave) ∧ (BKnight ⊕ BKnave).
-Same kind: A and B are the same type iff (AKnight ⇔ BKnight), i.e. Biconditional(AKnight, BKnight).
-Different kinds: A and B are different types iff exactly one is knight: AKnight ⊕ BKnight.
-A says “same kind”: AKnight ⇒ same_kind, AKnave ⇒ ¬same_kind.
-B says “different kinds”: BKnight ⇒ diff_kind, BKnave ⇒ ¬diff_kind.

KB: KB_2 = structure ∧ (AKnight ⇒ same_kind) ∧ (AKnave ⇒ ¬same_kind) ∧ (BKnight ⇒ diff_kind) ∧ (BKnave ⇒ ¬diff_kind).

Result: The only model has A a knave and B a knight. So **A is a knave, B is a knight.**

Puzzle 3:
(characters A, B, and C)

Statements:
A says either “I am a knight” or “I am a knave” (unknown which).
B says: “A said ‘I am a knave’” and “C is a knave.”
C says: “A is a knight.”

-Structure: Each of A, B, C is exactly one of knight/knave (three XOR constraints).
-“A said ‘I am a knave’”:** Logically, “A said ‘I am a knave’” is encoded as: (AKnight ⇒ AKnave) ∧ (AKnave ⇒ ¬AKnave). The second conjunct is AKnave ⇒ false, so this

whole expression is true only when A is neither knight nor knave — impossible. So the proposition “A said ‘I am a knave’” is **always false** in the puzzle world.
- **B’s full statement:** B says (that false proposition) ∧ CKnave, i.e. false ∧ CKnave = **false**. So B is asserting a false statement; hence B must be a knave (BKnight ⇒ false would force a contradiction). So **B is a knave.** We encode: BKnight ⇒ (B’s statement), BKnave ⇒ ¬(B’s statement).
- **C says “A is a knight”:** CKnight ⇒ AKnight, CKnave ⇒ ¬AKnight.

We do not add constraints from A’s statement (we don’t know which of the two sentences A actually said), so the KB does not uniquely determine A and C. The only symbol entailed is B is a knave.

Result: **B is a knave.** A and C are not determined by the KB (multiple models remain).

Problem III:

Part 1 – Natural Language to FOL:

a) $\forall_x \; number(x) \rightarrow \exists_y \; number(y) \wedge \neg(x = y)$

b) $\forall_x \; number(x) \rightarrow (odd(x) \wedge \neg even(x)) \vee (\neg odd(x) \wedge even(x))$

c) $\forall_x \; number(x) \wedge even(x) \rightarrow \exists_y \; number(y) \wedge succ(x,y) \wedge odd(y)$

d) $\forall_x \; number(x) \wedge odd(x) \rightarrow \exists_y \; number(y) \wedge succ(x,y) \wedge even(y)$

e) $\forall_x \; number(x) \rightarrow \exists_y \; number(y) \wedge Larger(y,x)$

f) $\forall_x \forall_y \forall_z \; number(x) \wedge number(y) \wedge number(z) \wedge Larger(x,y) \wedge Larger(y,z) \rightarrow Larger(x,z)$